AI 视频生成系统：完整技术文档与使用指南

目录

1. 系统概述与技术架构
2. 核心模型与组件
3. 算法实现详解
4. 论文引用与学术来源
5. 完整部署与使用指南
   - 系统要求
   - 项目获取与虚拟环境
   - 依赖安装（自动化脚本）
   - 预训练模型获取
   - 运行基础推理
   - WebUI 图形化界面
6. 内存/显存优化设置教程
7. 高级功能深度解析
8. 扩展开发指南
9. 加速技术使用指南（WebUI 版）
10. 开源与贡献指南
11. 许可与版权声明
12. 致谢与第三方模型许可

---

1. 系统概述与技术架构

本项目实现了一个基于**时空扩散状态机（TS-DSM）**与**多模态控制**的视频生成系统。它支持文本到视频、图生视频、音频驱动视频、镜头脚本控制等多种生成方式，并集成了物理约束、记忆注入、稀疏注意力、蒸馏加速等前沿技术，能够生成长达数分钟的高质量视频。系统采用模块化设计，各组件可独立替换或扩展，适合学术研究与实际应用。

1.1 系统架构图

```text
输入：文本/图像/音频/镜头脚本
    ↓
[多模态编码器] (CLIP / T5 / Wav2Vec2)
    ↓
[DiT 主干] ← [记忆库] ← [物理约束]
    ↓
[视频 VAE] (LeanVAE / AutoencoderKL)
    ↓
[后处理] (超分/插帧/风格/镜头运动)
    ↓
输出视频
```

1.2 核心技术借鉴

技术方向	借鉴工作	作用
扩散主干	DiT (Peebles & Xie, 2023)	Transformer 替代 U-Net，提升扩展性与
视频建模	Video Diffusion Models	3D 卷积 + 时空注意力
长视频生成	Self-Forcing / FAR	自回归训练范式
记忆与稀疏性	StoryMem / LongCat-Video	记忆注入 + 稀疏注意力，提升长视频一致
物理约束	NS-Diff / PhysDreamer	光流 + 纳维-斯托克斯方程，提升物理真

技术方向	借鉴工作	作用
偏好对齐	DPO (Rafailov et al., 2023)	后训练对齐物理偏好

---

2. 核心模型与组件

2.1 扩散主干：SpatialTemporalUNet (DiT)

- **结构**：基于 Transformer 的时空扩散模型，将视频潜变量分块后视为 token 序列，通过多层 Transformer 处理。
- **关键设计**：
  - 时空注意力：通过滑动窗口、块稀疏、步长稀疏等模式降低计算量。
  - 条件注入：文本、图像、音频等多模态特征通过交叉注意力融入 DiT 块。
  - KV 缓存与因果掩码：支持自回归生成。
- **参数规模**：1.3B（与 Wan2.1 系列兼容）。

2.2 视频 VAE

- **主要实现**：**LeanVAE**（由 westlake-repl/LeanVAE 提供），时间压缩 4 倍，空间压缩 8 倍，潜空间通道数 4。
- **备选实现**：Stability AI 的 **AutoencoderKL**（2D VAE 扩展为 3D），空间压缩 8 倍，无时间压缩，当 LeanVAE 不可用时自动回退。
- **预训练权重**：LeanVAE 权重存放于 `./models/LeanVAE`（需手动下载或自动获取），AutoencoderKL 存放于 `./models/vae`（由 diffusers 自动下载）。

2.3 多模态编码器

编码器	模型	作用
文本编码器	CLIP 文本编码器 / T5	文本描述编码为条件
图像编码器	CLIP 图像编码器	图生视频或多图参考
音频编码器	Wav2Vec2-base	音频特征提取
视频编码器	自研（基于 VAE 特征）	参考视频风格迁移

2.4 镜头控制器（LensController）

- **模型**：Transformer 编码器，将镜头脚本（JSON 格式）转换为隐向量。
- **支持参数**：镜头类型、相机运动、光影、色调、节奏等，可精细控制镜头语言。

2.5 记忆库与状态机（MemoryBank & State）

- **记忆库**：显式存储关键帧特征（通过交叉注意力注入模型），实现跨镜头角色/场景一致性。
- **状态机**：通过 `prev_state` 传递隐状态，保证相邻块间的光流连续性。

2.6 物理约束模块（PhysicsConstraint）

- **核心组件**：
  - RAFT 光流：计算速度场（若安装）。
  - 纳维-斯托克斯损失：约束流体运动。

- 刚体运动约束：最小急动度（Minimum-Jerk）。
- 碰撞检测：基于速度场散度。
- 弹性势能：基于形变梯度。

- **用途**：作为额外损失项训练模型，或在生成后评分（DPO）。

2.7 增强物理评分器（EnhancedPhysicsScorer）

- **作用**：融合多个物理指标，输出 0-1 分数，用于 DPO 数据筛选。
- **可选项**：可加载预训练物理判别器（如 PhysScore）。

2.8 后处理模块（Postprocess）

- 超分：Real-ESRGAN，提升分辨率。
- 帧插值：RIFE（可选），提高帧率。
- 镜头运动：仿射变换实现平移、缩放、旋转。
- 风格化：卡通、油画、素描等滤镜。
- 水印：文字叠加。

---

3. 算法实现详解

3.1 DiT 主干：时空扩散 Transformer

3.1.1 输入处理

给定视频潜变量 `x ∈ ℝ^(B×C×T×H×W)`，首先展平为序列：

```python
N = T * H * W
```

```
x_flat = x.permute(0,2,3,4,1).reshape(B, N, C)
x = self.input_proj(x_flat)  # 投影到模型维度 D
```

3.1.2 时间步与条件注入

时间步 `t` 通过正弦嵌入并经过 MLP 得到 `t_emb`；条件 `cond` 通过投影后与 `t_emb` 相加，再与 `x` 相加实现注入：

```python
t_emb = self._timestep_embedding(t, D)
x = x + t_emb.unsqueeze(1)
cond_emb = self.cond_proj(cond).mean(dim=1, keepdim=True)
x = x + cond_emb.expand(-1, N, -1)
```

3.1.3 核心 Transformer 层（DiTBlock）

每个块包含：

- **归一化 + 稀疏自注意力**：支持滑动窗口、块稀疏、步长稀疏等模式。
- **记忆交叉注意力**（若启用）：从外部记忆库检索特征并注入。
- **条件交叉注意力**（可选）：将条件嵌入作为键值。
- **MLP 前馈网络**：4 倍扩展。

3.1.4 滑动窗口注意力实现（三区缓存）

为了支持长序列，实现了**三区缓存**：固定首帧区（sink）、可压缩的中间区（mid）、最近帧区（recent）。

- 当 `recent` 超过阈值，将溢出的部分送入 `mid_raw_buffer`，累积到一定量后触发压缩（通过可学习压缩器或分层池化）。
- 注意力计算时，拼接 `sink + compressed_mid + recent + current`，并应用窗口掩码（仅保留窗口内及因果关系）。

3.1.5 输出

最后通过输出投影层将特征映射回原始通道数，并重塑为视频形状 `(B,C,T,H,W)`。

3.2 记忆库（MemoryBank）

- **存储结构**：一个固定大小的矩阵 `memory ∈ ℝ^(1×M×D)`，以及对应的年龄和重要性缓冲。
- **写入**：当新特征到来时，选择最老且最不重要的槽位，用动量更新：

  ```python
  memory[oldest] = alpha * memory[oldest] + (1-alpha) * new_feat
  ```

- **读取**：基于余弦相似度或可学习检索器，返回 top-k 相关记忆。
- **对比学习**（可选）：使用 InfoNCE 损失拉近同一视频不同片段的特征。

3.3 物理约束损失

3.3.1 光流计算（RAFT）

利用 RAFT 模型计算相邻帧的光流场 `flow ∈ ℝ^(B×2×(T-1)×H×W)`。

3.3.2 纳维-斯托克斯损失

根据速度场 `u, v` 计算：

```text
du_dt + u·∇u - v∇²u = 0
```

采用有限差分近似，约束非线性和扩散项。

3.3.3 刚体损失（Minimum-Jerk）

对于给定的轨迹 `traj`，计算加速度的二阶差分：

```text
jerk = traj[t+2] - 2*traj[t+1] + traj[t]
```

约束急动度最小化。

3.4 蒸馏加速（TeaCache & 4-步蒸馏）

3.4.1 TeaCache

基于 timestep embedding 的特征相似度判断是否重用上一步的预测结果：

- 提取当前步的输入特征（例如 latents 的通道均值）。
- 与缓存的特征计算 L1 距离，若小于阈值则复用缓存的 noise_pred。
- 阈值可调（0.1~0.3），典型加速 30-50%。

3.4.2 4-步蒸馏（DMD + 对抗）

- **DMD**：最小化学生模型与教师模型预测的 MSE。
- **对抗损失**：判别器区分学生输出与真实数据。
- 训练后，推理步数可降至 4 步，速度提升 12 倍。

3.5 自回归生成与 KV 缓存

在自回归模式下，模型逐帧生成：

1. 第一帧随机初始化或给定。

2. 对于后续帧，将前序帧的 KV 缓存起来，在注意力计算时使用缓存的历史信息。
3. 每一步只生成下一帧的潜变量，并通过 VAE 解码为像素帧。

3.6 分块融合（TileGenerator）

高分辨率生成时，将画面切分为多个重叠的 tile：

1. 每个 tile 独立生成（利用并行批处理）。
2. 融合时，每个 tile 边缘使用高斯权重衰减，然后加权平均合并。

---

4. 论文引用与学术来源

4.1 可确认的论文引用

```
bibtex
@inproceedings{peebles2023dit,
  title={Scalable Diffusion Models with Transformers},
  author={Peebles, William and Xie, Saining},
  booktitle={ICCV},
  year={2023}
}
@inproceedings{rombach2022high,
  title={High-Resolution Image Synthesis with Latent Diffusion Models},
  author={Rombach, Robin and others},
  booktitle={CVPR},
  year={2022}
}
@inproceedings{radford2021learning,
  title={Learning Transferable Visual Models From Natural Language Supervisi
on},
  author={Radford, Alec and others},
```

```
  booktitle={ICML},
  year={2021}
}
@article{baevski2020wav2vec,
  title={wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Rep
resentations},
  author={Baevski, Alexei and others},
  journal={NeurIPS},
  year={2020}
}
@article{rafailov2023dpo,
  title={Direct Preference Optimization: Your Language Model is Secretly a R
eward Model},
  author={Rafailov, Rafael and others},
  journal={arXiv:2305.18290},
  year={2023}
}
```

4.2 参考技术方向

- 稀疏注意力：LongCat-Video / Block Sparse Attention video generation
- 记忆注入：StoryMem / Memory bank video generation
- 物理约束：NS-Diff / PhysDreamer
- 物理评分：PhysBench / PhysScore
- 蒸馏加速：DMD diffusion distillation / LCM video generation

---

5. 完整部署与使用指南

5.1 系统要求

创意版（入门/个人尝鲜）

组件	配置	说明
处理器 (CPU)	Intel i5 / AMD Ryzen 5（4 核）	满足基础运算
内存 (RAM)	16 GB	可支持 ≤720p 短视频生成
硬盘 (存储)	512 GB SSD	可容纳模型、依赖及少量临时文件
GPU	无（CPU 模式）或 GTX 1060 6GB	CPU 模式仅限 256p 短时测试；若有 6GB 显 480p/10 秒视频
Python	3.9	
操作系统	Windows 10/11 / Ubuntu 20.04	

适用场景：分辨率 ≤480p，时长 ≤10 秒，帧率 ≤24fps，创意验证、快速原型。

专业版（标准工作站/1080p 生产）

组件	配置	说明
处理器 (CPU)	Intel i7 / AMD Ryzen 7（8 核）	提升数据处理与后处理速度
内存 (RAM)	32 GB	支持 1080p 中等时长视频，避免交换
硬盘 (存储)	1 TB NVMe SSD	满足模型、依赖及大量临时视频文件

组件	配置	说明
GPU	NVIDIA RTX 4080 (16 GB) 或同级别	可流畅生成 1080p/30 秒/24fps 视频，支持成
Python	3.10	
操作系统	Windows 11 / Ubuntu 22.04	

适用场景：分辨率 720p–1080p，时长 ≤30 秒，帧率 ≤30fps，中高质量视频生产。

服务器版（高性能/长视频/4K/8K）

组件	配置	说明
处理器 (CPU)	Intel Xeon / AMD EPYC（16 核以上）	多核处理并行任务、光流计算等
内存 (RAM)	64 GB – 128 GB	支持超长视频处理与多任务并发
硬盘 (存储)	2 TB+ NVMe SSD（可扩展）	容纳大量中间文件和最终视频
GPU	NVIDIA RTX 4090 (24 GB) × 多卡 或 A100 (40/80 GB)	支持 4K/8K 长视频、自回归生 并行
Python	3.10	
操作系统	Ubuntu 22.04 / Windows Server	服务器操作系统

适用场景：分辨率 4K/8K，时长 ≥1 分钟，帧率 ≥60fps，多用户并发，模型训练。

5.2 项目获取与虚拟环境

```bash
# 解压项目到 D:\video_gen
cd D:\video_gen
python -m venv venv
venv\Scripts\activate
```

5.3 依赖安装（自动化脚本）

创建 `setup.bat`（Windows）并运行：

```batch
@echo off
chcp 65001 >nul
title AI 视频生成系统 - 一键安装
echo ==================================================
echo AI 视频生成系统 完整依赖安装脚本
echo ==================================================
echo.
python --version >nul 2>&1
if errorlevel 1 (
 echo [错误] 未找到 Python，请先安装 Python 3.9-3.11。
 pause
 exit /b 1
)
echo [√] Python 已安装
if not exist "venv" (
 echo [信息] 创建虚拟环境...
 python -m venv venv
) else (
 echo [信息] 虚拟环境已存在
)
call venv\Scripts\activate.bat
python -m pip install --upgrade pip setuptools wheel -i https://pypi.tuna.ts
inghua.edu.cn/simple
:: 检测 CUDA
python -c "import torch; print(torch.cuda.is_available())" 2>nul
if errorlevel 1 (
 echo [信息] 未检测到 CUDA，安装 CPU 版 PyTorch
```

```
 pip install torch torchvision torchaudio -i https://pypi.tuna.tsinghua.edu.
cn/simple
) else (
 echo [信息] 检测到 CUDA，安装 GPU 版 PyTorch (CUDA 12.1)
 pip install torch torchvision torchaudio --index-url https://download.pytor
ch.org/whl/cu121 -i https://pypi.tuna.tsinghua.edu.cn/simple
)
pip install -r requirements.txt -i https://pypi.tuna.tsinghua.edu.cn/simple
pip install xformers realesrgan basicsr -i https://pypi.tuna.tsinghua.edu.cn
/simple
:: 可选：RAFT
where git >nul 2>&1
if not errorlevel 1 (
 if not exist "RAFT" git clone https://github.com/princeton-vl/RAFT.git
 cd RAFT && pip install -e . && cd ..
)
echo.
echo ==================================================
echo 依赖安装完成！请下载预训练模型
echo ==================================================
echo 运行：python -c "from huggingface_hub import hf_hub_download; import o
s; os.makedirs('./models', exist_ok=True); hf_hub_download(repo_id='Wan-AI/
Wan2.1-1.3B', filename='model.safetensors', local_dir='./models')"
pause
```

5.4 预训练模型获取

- **核心扩散模型（Wan2.1-1.3B）**：自动下载（推荐）：

  bash

  ```
  python -c "from huggingface_hub import hf_hub_download; import os; os.makedi
  rs('./models', exist_ok=True); hf_hub_download(repo_id='Wan-AI/Wan2.1-1.3B
  ', filename='model.safetensors', local_dir='./models')"
  ```

- **视频 VAE（LeanVAE）**：系统会在首次使用时自动从 Hugging Face 下载，

  也可手动放入 `./models/LeanVAE`（目录结构需包含 `model.ckpt` 等文件）。

  若 LeanVAE 不可用，将回退到 AutoencoderKL（无需额外下载）。

5.5 运行基础推理

- **随机权重测试**（验证环境）：

  bash

  ```
  python main.py --mode infer --prompt "a cat" --output test.mp4
  ```

- **使用预训练权重**：

  bash

  ```
  python main.py --mode infer --prompt "a cat" --output cat.mp4 --pretrained ./models/model.safetensors
  ```

常用参数说明

参数	默认	说明
`--duration`	2.0	时长（秒）
`--fps`	8	帧率
`--resolution`	"256p"	分辨率（256p,480p,720p,1080p 等）
`--steps`	50	扩散步数
`--cfg_scale`	7.5	提示词强度
`--style`	None	风格（cartoon,oil,sketch,sepia）
`--camera`	None	镜头运动（pan,tilt,zoom,rotate）

5.6 WebUI 图形化界面

启动命令：

bash

```
python main.py --mode webui
```

浏览器访问 http://127.0.0.1:7860。

WebUI 界面说明：

- **左侧参数区**：核心参数、高级设置（折叠）、多模态输入、批量生成与任务面板、模型与库、风格与镜头。
- **右侧预览区**：视频播放器、任务列表、配置管理、模式切换、加速选项。
- **使用提示**：所有加速选项均可在 WebUI 中勾选，系统会根据硬件条件智能推荐（智能模式）。建议新手先使用“智能推荐”或“应用预设”，逐步调整参数。

---

6. 内存/显存优化设置教程

6.1 CPU 模式（无 NVIDIA 显卡）

核心原则：极低分辨率、极短时长、极低帧率、使用分块生成。

```bash
python main.py --mode infer --prompt "a cat" --duration 2 --fps 8 --resolution 256p --pretrained ./models/model.safetensors
```

注意：CPU 模式下，潜在空间 `N=4*32*32=4096`，内存占用安全。如需生成长视频，必须使用 `generate_long` 方法（WebUI 自动调用）。CPU 模式下建议避免长视频。

6.2 GPU 模式（有 NVIDIA 显卡）

根据显存容量选择参数：

显存	推荐参数
8GB	`--duration 5 --fps 24 --resolution 480p --steps 25`
16GB	`--duration 10 --fps 24 --resolution 720p --steps 50`
24GB+	`--duration 30 --fps 24 --resolution 1080p --steps 50`

- **自回归生成**（需≥16GB 显存）：

```bash
python main.py --mode infer --prompt "a cat walking" --duration 60 --resolution 720p --pretrained ./models/model.safetensors
```

- **开启半精度推理**（减少显存）：在 `inferencer.py` 的 `__init__` 中添加 `self.model = self.model.half()`。

6.3 模型优化建议

- 降低推理步数：`--steps 20`（速度提升 2.5 倍，画质损失小）
- 使用蒸馏模型：下载 `lightx2v/Wan2.1-T2V-1.3B-StepDistill`，步数设为 4
- 调整分块大小：`config.py` 中 `max_block_frames = 16`
- 关闭物理约束：`use_raft_physics = False`

6.4 常见问题排查

错误	原因	解决
`RuntimeError: not enough memory`	序列过长	降低 duration/fps/resolution

错误	原因	解决
`CUDA out of memory`	显存不足	降低分辨率/步数，启用半精度
生成速度极慢	CPU 模式	安装 GPU 版 PyTorch

---

7. 高级功能深度解析

7.1 自回归长视频生成

- **原理**：因果注意力掩码 + KV 缓存，逐帧预测。
- **使用**：`--ar_mode` 参数（需代码支持），或直接调用 `inferencer.generate_ar`。
- **要求**：显存≥16GB。

7.2 记忆注入（角色/场景一致性）

- **原理**：MemoryBank 存储关键帧特征，通过交叉注意力注入。
- **启用**：默认集成，无需额外参数。

7.3 物理偏好对齐（DPO 微调）

- **步骤**：

1. 生成正负样本对：`trainer.generate_dpo_data(inferencer)`
2. 微调：`trainer.train_dpo_step`

- **效果**：提升物理真实性（重力、碰撞等）。

7.4 稀疏注意力与长序列优化

- **模式**：滑动窗口、块稀疏、步长稀疏（在 `config.py` 中修改 `attn_type`）。
- **效果**：支持数分钟长序列推理，显存占用线性增长。

7.5 4K/8K 超分生成

- **使用**：`--resolution 4k` 或 `8k`。
- **原理**：TileGenerator 自动分块生成后融合。

---

8. 扩展开发指南

8.1 添加新数据集

1. 在 `data/` 下创建 `my_dataset.py`，继承 `torch.utils.data.Dataset`。
2. 实现 `__len__` 和 `__getitem__`，返回字典 `{'video': tensor, 'text': str, ...}`。
3. 在 `data/dataset.py` 的 `get_dataset` 中加入分支。

8.2 集成新编码器

1. 在 `models/` 下创建编码器类。
2. 在 `main.py` 中根据配置加载。
3. 在 `trainer.py` 和 `inferencer.py` 中处理输出。

8.3 添加后处理模块

1. 在 `postprocess/` 下创建新模块。
2. 在 `postprocess/__init__.py` 中导出。
3. 在 `postprocess/postprocess.py` 的 `postprocess_video` 函数中调用。

---

9. 加速技术使用指南（WebUI 版）

WebUI 界面中提供了多种加速选项，用户可根据硬件条件自由组合：

选项	作用	推荐场景	WebUI 对应控件
TurboQuant	对 KV 缓存进行极坐标量化和 QJL 校正，压缩显存约 50%	任何长视频或高分辨率生成	右侧“TurboQuant”复选框
二次蒸馏	将推理步数强制降至 4，速度提升约 12 倍	追求速度、模型已蒸馏时	右侧“二次蒸馏”复选框（自动限制步数 ≤4）
金字塔采样	先低分辨率生成，再逐步细化，减少高分辨率计算量	4K/8K 或超长视频	右侧“金字塔采样”复选框
并行 tile	将高分辨率图像分块并行生成，加速	显存充足时（≥16GB）	右侧“并行 tile”复选框，下方可设置 tile

选项	作用	推荐场景	WebUI 对应控件
	2-5 倍		批次数
TensorRT	使用 TensorRT 引擎加速模型推理（需预编译引擎）	对延迟有极致要求时	右侧“TensorRT”复选框，需指定引擎路径

- **智能推荐模式**：点击“智能推荐”按钮，系统会根据当前分辨率、时长、显存等信息自动勾选合适的加速选项，建议新手使用。
- **全开模式**：一键启用所有加速技术，适合追求极致性能且显存充足的用户。

使用示例：

1. 在 WebUI 左侧设置提示词为“a cat running in a cyberpunk city, 4k”。
2. 时长设置为 60 秒，帧率 30，分辨率 4K。
3. 点击右侧“智能推荐”，自动勾选“TurboQuant”“金字塔采样”“并行 tile”等。
4. 点击“开始生成视频”，任务面板会显示进度和预计剩余时间。

注意：加速选项并非越多越好，需根据显存和模型支持情况选择。例如，若模型未经过蒸馏训练，勾选“二次蒸馏”可能导致画质严重下降。

---

10. 开源与贡献指南

10.1 环境准备

- 安装 Python 3.9-3.11

- 推荐使用 NVIDIA GPU（显存 ≥8GB）
- 至少 20GB 硬盘空间用于存放模型和依赖

10.2 获取代码

```bash
git clone https://github.com/your-repo/video_gen.git
cd video_gen
```

10.3 安装依赖（库安装情况）

库名	安装方式	可选性	许可证	说明
Wav2Vec2	`pip install transformers`（已包含）	必需（音频编码）	Apache 2.0	由 transformers 库提供，需保留版权声明
AutoencoderKL	`pip install diffusers`（已包含）	必需（VAE 备选）	MIT	由 diffusers 库提供
LeanVAE	需从 Hugging Face 下载权重	可选（主要 VAE）	*需自行确认*	见第 12 节
Real-ESRGAN	`pip install realesrgan basicsr`	可选（超分）	BSD-3-Clause	若启用超分后处理则需要，已在脚本中
RAFT	`git clone ... && pip install -e .`	可选（物理约束）	BSD-3-Clause	若启用光流物理约束则需要，不安装则使用简化光流

库名	安装方式	可选性	许可证	说明
xFormers	`pip install xformers`	可选（加速）	BSD-3-Clause	可显著降低显存并加速注意力计算
Hugging Face Transformers	`pip install transformers`（已包含）	必需（模型加载）	Apache 2.0	用于加载 CLIP、Wav2Vec2 等模型

一键安装命令（Windows）：参见第 5.3 节的 `setup.bat` 脚本。

10.4 下载预训练模型

```bash
# 下载核心扩散模型
python -c "from huggingface_hub import hf_hub_download; import os; os.makedirs('./models', exist_ok=True); hf_hub_download(repo_id='Wan-AI/Wan2.1-1.3B', filename='model.safetensors', local_dir='./models')"
# LeanVAE 会在首次使用时自动下载，也可手动放置
```

10.5 快速开始

```bash
# 启动 WebUI
python main.py --mode webui

# 或直接通过命令行生成
python main.py --mode infer --prompt "a cat" --output cat.mp4 --pretrained ./models/model.safetensors
```

10.6 贡献代码

欢迎提交 Issue 和 Pull Request。请确保代码风格符合 PEP 8，并在提交前测试通过。

10.7 开源协议

本项目采用 **Apache License 2.0** 开源协议，使用前请阅读 LICENSE 文件。

---

11. 许可与版权声明

11.1 开源许可

本项目整体采用 **Apache License 2.0** 开源协议。使用本项目时需保留原始版权声明。

11.2 第三方模型许可

详见 第 12 节：致谢与第三方模型许可。

11.3 免责声明

本代码仅供学习和研究使用。使用第三方模型时，请遵守各模型的许可证条款。因使用本代码产生的任何法律问题，项目作者不承担任何责任。

---

12. 致谢与第三方模型许可

本系统能够实现高质量的文本到视频生成，离不开开源社区提供的优秀模型与框架。我们在此向以下项目及贡献者表示诚挚感谢，并整理第三方模型的许可信息，请使用者遵守相应条款。

第三方模型许可一览表

组件 / 模型	许可证	商用许可	核心义务	说明
PyTorch	BSD-style	✅ 允许	保留版权声明	核心深度学习框架
Torchvision	BSD	✅ 允许	保留版权声明	图像/视频处理工具集
Transformers	Apache 2.0	✅ 允许	保留版权声明及 NOTICE 文件	Hugging Face 模型加载库
Diffusers	Apache 2.0	✅ 允许	保留版权声明及 NOTICE 文件	扩散模型工具库
Accelerate	Apache 2.0	✅ 允许	保留版权声明	分布式训练/推理工具
Gradio	Apache 2.0	✅ 允许	保留版权声明	WebUI 界面库

组件 / 模型	许可证	商用许可	核心义务	说明
FastAPI & Uvicorn	MIT	✅ 允许	保留版权声明	API 服务框架
OpenCV-Python	BSD	✅ 允许	保留版权声明	计算机视觉处理
NumPy / SciPy	BSD	✅ 允许	保留版权声明	数值计算
tqdm	MIT	✅ 允许	保留版权声明	进度条库
scikit-image	BSD	✅ 允许	保留版权声明	图像处理工具
CLIP	MIT	✅ 允许	保留版权声明	OpenAI 多模态模型，用于文本/图像编码
Wav2Vec2	Apache 2.0	✅ 允许	保留版权声明	Meta 语音表示学习模型，用于音频编码
AutoencoderKL	MIT	✅ 允许	保留版权声明	Stability AI 的 VAE，作为 LeanVAE 的备选

组件 / 模型	许可证	商用许可	核心义务	说明
LeanVAE	*需自行确认*	?	根据来源仓库的许可证	主要 VAE（时间压缩 4 倍），代码中引用自 `westlake-repl/LeanVAE`，其 Hugging Face 仓库未明确标注许可证。用户若使用，需自行查阅其官方仓库并遵守相应条款。
Wan2.1-1.3B	Apache 2.0	✅ 允许	保留版权声明及 NOTICE 文件	阿里开源的 1.3B 视频生成模型，与 DiT 结构兼容
Real-ESRGAN	BSD-3-Clause	✅ 允许	保留版权声明	腾讯开源超分模型，用于视频后处理
basicsr	Apache 2.0	✅ 允许	保留版权声明	Real-ESRGAN 依赖的基础图像恢复库
RAFT	BSD-3-Clause	✅ 允许	保留版权声明	普林斯顿大学光流计算库，用于物理约束
xFormers	BSD-3-Clause	✅ 允许	保留版权声明	优化注意力计算，显著降低显存占用

组件 / 模型	许可证	商用许可	核心义务	说明
DeepSpeed（可选)	Apache 2.0	✅ 允许	保留版权声明	大规模训练优化库
Torchaudio	BSD	✅ 允许	保留版权声明	音频处理库
librosa (可选)	ISC	✅ 允许	保留版权声明	音频特征提取库

使用建议

在您的项目中，请按以下格式注明来源：

本项目使用了以下开源模型：Wan2.1-1.3B (Alibaba, Apache 2.0), CLIP (OpenAI, MIT), Wav2Vec2 (Meta, Apache 2.0), AutoencoderKL (Stability AI, MIT), LeanVAE (westlake-repl，许可证请查阅其仓库), Real-ESRGAN (Tencent, BSD-3-Clause), RAFT (Princeton University, BSD-3-Clause）等。

文档结束
如需进一步了解代码细节，请查阅项目源码及注释。